

Problem 2 (Uphill paths), Cell 1: $U(Q_3)$ and $U(Q_4)$

Team submission

BAINSA Hackathon, 26 September 2026

1 Cell 1

Official setting.

Let G be a finite simple graph on n vertices. A *labelling* of G is a bijection f from $V(G)$ to $\{1, 2, \dots, n\}$. Fix a labelling. A vertex v is a *valley* if every neighbour w of v has $f(w) > f(v)$; an isolated vertex counts as a valley. An *uphill path* is a sequence $(v_1, v_2, \dots, v_k)$ with $k \geq 1$ in which v_1 is a valley, v_i and v_{i+1} are adjacent for each i , and $f(v_1) < f(v_2) < \dots < f(v_k)$. A valley on its own ($k = 1$) counts as an uphill path. Let $U(G)$ be the smallest possible number of uphill paths, taken over all labellings of G . (IMO 2022 Problem 6 asks for U of the $n \times n$ grid graph; the answer is $2n^2 - 2n + 1$.) The d -dimensional hypercube Q_d has vertex set $\{0, 1\}^d$, and two vertices are adjacent when they differ in exactly one coordinate; Q_d has 2^d vertices and $d \cdot 2^{d-1}$ edges. For C1 to C4, hand in the values and, for each value, an explicit labelling that attains it, as a list of the 2^d vertices in increasing label order, written as 0/1 strings.

Task. Determine $U(Q_3)$ and $U(Q_4)$, and give a labelling attaining each value.

Status. **Solved**, by hand. No computation is needed; the labellings were also counted mechanically (`code/uphill_eval.py`).

Answer and where it is proved.

- $U(Q_3) = 14$ and $U(Q_4) = 34$ (Theorem 1.8); labellings in §1.8.
- The proof rests on an exact formula for the number of uphill paths (Proposition 1.3). It gives $U(Q_d) \geq 2^d + (d-1)\nabla(Q_d)$ for every d , where ∇ is the decycling number (Theorem 1.4). In the other direction, every even-weight code with minimum distance 4 gives a labelling (Proposition 1.6).
- Sections 1.1–1.6 hold for every d and are used by all the later cells of this problem.

1.1 Counting uphill paths

Fix a graph G and a labelling f . A *lower neighbour* of v is a neighbour w with $f(w) < f(v)$, and a *higher neighbour* is a neighbour w with $f(w) > f(v)$. Write $\text{out}(v)$ for the number of higher neighbours of v . So v is a valley if and only if it has no lower neighbour. Let $N(v) = N_f(v)$ be the number of uphill paths whose last vertex is v , and $P(f) = \sum_v N(v)$ the number of uphill paths. Then $U(G) = \min_f P(f)$.

Lemma 1.1 (Recursion). $N(v) = 1$ if v is a valley, and $N(v) = \sum_w N(w)$, summed over the lower neighbours w of v , otherwise. In particular $N(v) \geq 1$ for every v .

Proof. If $(v_1, \dots, v_k)$ is an uphill path with $v_k = v$ and $k \geq 2$, then v_{k-1} is a lower neighbour of v and $(v_1, \dots, v_{k-1})$ is an uphill path ending at v_{k-1} . Conversely, if w is a lower neighbour of v , then every uphill path ending at w extends by v to an uphill path ending at v . These two maps are inverse to each other. The one-vertex sequence (v) is an uphill path if and only if v is a valley, and a valley has no lower neighbours. The inequality $N(v) \geq 1$ follows by induction on $f(v)$: a non-valley has at least one lower neighbour. $\square$

1.2 Vertices with a single uphill path

Put $S = S_f = \{v : N(v) = 1\}$ and $R = R_f = \{v : N(v) \geq 2\}$. Write $G[X]$ for the subgraph induced by X , and $e(X)$ for its number of edges.

Lemma 1.2. (a) *If w is a lower neighbour of v , then $N(v) \geq N(w)$. A vertex of S that is not a valley has exactly one lower neighbour, and it lies in S .*

(b) *Every edge between $s \in S$ and $u \in R$ satisfies $f(s) < f(u)$. Every higher neighbour of a vertex of R lies in R .*

(c) *$G[S]$ is a forest. Every valley of G lies in S , and every component of $G[S]$ contains exactly one valley.*

Proof. (a) By Lemma 1.1, $N(v)$ is a sum of terms ≥ 1 that includes $N(w)$. If $v \in S$ is not a valley, then this sum equals 1, so it has a single term, equal to 1.

(b) If $f(u) < f(s)$, then u is a lower neighbour of s and (a) gives $N(s) \geq N(u) \geq 2$, a contradiction. Likewise, a higher neighbour v of $u \in R$ has $N(v) \geq N(u) \geq 2$.

(c) Valleys have $N = 1$. Every edge of $G[S]$ has a higher end v , and by (a) the lower end is the unique lower neighbour of v . So $e(S)$ equals the number of non-valleys in S . Let K be a component of $G[S]$, with n_K vertices. Its vertex of smallest label has no lower neighbour in K . By (a) it has no lower neighbour in G at all, so it is a valley. Counting as above inside K , $e(K) = n_K - (\text{number of valleys in } K) \leq n_K - 1$. A connected graph has $e(K) \geq n_K - 1$. Hence K is a tree and contains exactly one valley. $\square$

1.3 An exact formula

Proposition 1.3 (Exact formula). *For every graph G and every labelling f ,*

$$P(f) = |V(G)| + \sum_{u \in R} (\deg u - 1) + \sum_{u \in R} (N(u) - 2) \text{out}(u).$$

All terms of the last sum are ≥ 0 . In particular, if G is d -regular, then $P(f) \geq |V(G)| + (d-1)|R_f|$.

Proof. Sum the recursion of Lemma 1.1 over all v . Each pair $(w, \text{higher neighbour } v \text{ of } w)$ contributes $N(w)$ once, so

$$P(f) = \#\{\text{valleys}\} + \sum_w N(w) \text{out}(w).$$

Let c be the number of components of $G[S]$. By Lemma 1.2(c), there are c valleys and $e(S) = |S| - c$. For $w \in S$ we have $N(w) = 1$. Moreover, $\sum_{w \in S} \text{out}(w) = e(S) + e(S, R)$, because each edge inside S is counted at its lower end, each S - R edge is counted at its end in S (Lemma 1.2(b)), and edges inside R are not counted. For $w \in R$, all higher neighbours lie in R (Lemma 1.2(b)), so $\sum_{w \in R} \text{out}(w) = e(R)$. Using $e(S, R) = \sum_{u \in R} \deg u - 2e(R)$, we get

$$P(f) = c + (|S| - c) + e(S, R) + \sum_{u \in R} N(u) \text{out}(u) = |S| + \sum_{u \in R} \deg u + \sum_{u \in R} (N(u) - 2) \text{out}(u),$$

and $|S| = |V(G)| - |R|$. Finally $N(u) \geq 2$ on R . $\square$

1.4 Decycling sets

A set $R \subseteq V(G)$ is a *decycling set* (or feedback vertex set) if $G - R = G[V \setminus R]$ is a forest. The *decycling number* $\nabla(G)$ is the smallest size of a decycling set. By Lemma 1.2(c), R_f is a decycling set for every labelling f .

Theorem 1.4 (Lower bound). *If G is d -regular, then $U(G) \geq |V(G)| + (d-1)\nabla(G)$. In particular, $U(Q_d) \geq 2^d + (d-1)\nabla(Q_d)$ for every $d \geq 1$.*

Proof. Proposition 1.3 and $|R_f| \geq \nabla(G)$. □

Proposition 1.5 (Upper bound). *Let G be d -regular and let R be a decycling set that is independent (no edge inside R). Then some labelling has exactly $|V(G)| + (d-1)|R|$ uphill paths. Consequently, if Q_d has an independent decycling set of size $\nabla(Q_d)$, then $U(Q_d) = 2^d + (d-1)\nabla(Q_d)$.*

Proof. Give the smallest labels to $V \setminus R$, tree by tree. Inside each tree of the forest $G - R$, use breadth-first order from a root. Then give the remaining labels to R , in any order. The neighbours of a vertex $v \notin R$ are its tree neighbours and some vertices of R , and the latter come later. In breadth-first order, a non-root vertex comes after its parent and before its children. So a root is a valley, and a non-root vertex has exactly one lower neighbour, its parent. By induction along the order, $N(v) = 1$ for all $v \notin R$. A vertex $u \in R$ has all its d neighbours outside R , all labelled earlier, so $N(u) = d$. The total is $(|V| - |R|) + d|R|$. The last sentence then follows from Theorem 1.4. □

1.5 Code labellings

For $x \in \{0, 1\}^d$, let $|x|$ be its weight (number of ones). Let E_d and O_d be the sets of vertices of even and of odd weight. Every edge of Q_d joins E_d to O_d , so both sets are independent and have size 2^{d-1} . The distance $|x \oplus y|$ of two vertices of E_d is even.

Proposition 1.6 (Code labelling). *Let $d \geq 2$, and let $C \subseteq E_d$ have pairwise distances ≥ 4 . Consider the labelling L_C that lists first the vertices of C , then all of O_d , then $E_d \setminus C$ (each block in any order). It has exactly*

$$P(L_C) = 2^d + (d-1)(2^{d-1} - |C|)$$

uphill paths. Moreover, $E_d \setminus C$ is an independent decycling set of Q_d . Hence $\nabla(Q_d) \leq 2^{d-1} - |C|$ and $U(Q_d) \leq 2^d + (d-1)(2^{d-1} - |C|)$.

Proof. We use Lemma 1.1. A codeword has only odd neighbours, all labelled later, so it is a valley: $N = 1$. The earlier neighbours of an odd vertex x are exactly its neighbours in C . Two of them would be at distance 2 from each other, so there is at most one. Hence $N(x) = 1$: either x is a valley, or it has a single lower neighbour, with $N = 1$. A vertex $y \in E_d \setminus C$ has d odd neighbours, all labelled earlier, so $N(y) = d$. Summing, $P = |C| + 2^{d-1} + d(2^{d-1} - |C|)$. In this labelling $R_{L_C} = E_d \setminus C$ (as $d \geq 2$), which is decycling by Lemma 1.2(c) and independent since it lies in E_d . □

1.6 A counting bound for the decycling number

Lemma 1.7. *Let G be d -regular with n vertices and m edges, and let $R \neq V(G)$ be a decycling set. Let c be the number of components of $G - R$. Then*

$$(d-1)|R| = m - n + c + e(R) \geq m - n + 1.$$

For Q_d this reads $(d-1)|R| \geq (d-2)2^{d-1} + 1$.

Proof. $G - R$ is a forest with $n - |R|$ vertices and c components, so it has $n - |R| - c$ edges. The edges meeting R number $d|R| - e(R)$. Adding up gives $m = n - |R| - c + d|R| - e(R)$. □

1.7 Q_3 and Q_4

Theorem 1.8. $U(Q_3) = 14$ and $U(Q_4) = 34$. Also $\nabla(Q_3) = 3$ and $\nabla(Q_4) = 6$.

Proof. $d = 3$: Lemma 1.7 gives $2\nabla(Q_3) \geq 5$, so $\nabla(Q_3) \geq 3$, and Theorem 1.4 gives $U(Q_3) \geq 8 + 2 \cdot 3 = 14$. The code $C = \{000\}$ in Proposition 1.6 gives a labelling with $8 + 2 \cdot (4 - 1) = 14$ uphill paths and a decycling set of size 3.

$d = 4$: Lemma 1.7 gives $3\nabla(Q_4) \geq 17$, so $\nabla(Q_4) \geq 6$ and $U(Q_4) \geq 16 + 3 \cdot 6 = 34$. The code $C = \{0000, 1111\}$ (distance 4) gives $16 + 3 \cdot (8 - 2) = 34$ and a decycling set of size 6. $\square$

Remark 1.9. An exhaustive check over all $8! = 40320$ labellings of Q_3 (`uphill_eval.py --brute 3`) also returns 14. The proof above does not use it. For $d = 2$ the same argument gives $U(Q_2) = U(C_4) = 5$. In words, Proposition 1.3 says that an optimal labelling must make few vertices “expensive” ($N \geq 2$), and those vertices must still break every cycle. For an arbitrary graph, the edge count in the proof of Lemma 1.7 reads $|E| = |S| - c + \sum_{u \in R} \deg u - e(R)$. Combined with Proposition 1.3, it gives $P(f) \geq |E| + c + e(R) \geq |E| + 1$. This is the lower bound $2n^2 - 2n + 1$ of the grid problem [3].

1.8 The labellings

Vertices in increasing label order (the number before each row is the label of its first entry). The labellings are L_C of Proposition 1.6: first the codewords, then the odd vertices, then the remaining even vertices, each block in increasing binary order.

Q_3 , $C = \{000\}$, 14 uphill paths:

1: 000 001 010 100 111 011 101 110

Q_4 , $C = \{0000, 1111\}$, 34 uphill paths:

1: 0000 1111 0001 0010 0100 0111 1000 1011
9: 1101 1110 0011 0101 0110 1001 1010 1100

Both counts were confirmed with the exact evaluator `code/uphill_eval.py`, which applies Lemma 1.1 directly.

References

- [1] S. Bau, L. W. Beineke, The decycling number of graphs, *Australas. J. Combin.* 25 (2002) 285–298.
- [2] P. Focardi, F. L. Luccio, D. Peleg, Feedback vertex set in hypercubes, *Inform. Process. Lett.* 76 (2000) 1–5.
- [3] International Mathematical Olympiad 2022, Problem 6 (uphill paths in a Nordic square).
- [4] F. J. MacWilliams, N. J. A. Sloane, *The Theory of Error-Correcting Codes*, North-Holland, 1977.
- [5] D. A. Pike, Decycling hypercubes, *Graphs Combin.* 19 (2003) 547–550.